Transformer Attention & Explainability (Amazon Polarity)

An End-to-End NLP Pipeline and Comparative XAI Analysis

Name: Muhammad Subhan Shahid

Roll Number: 23F-6109

Program: BS Artificial Intelligence

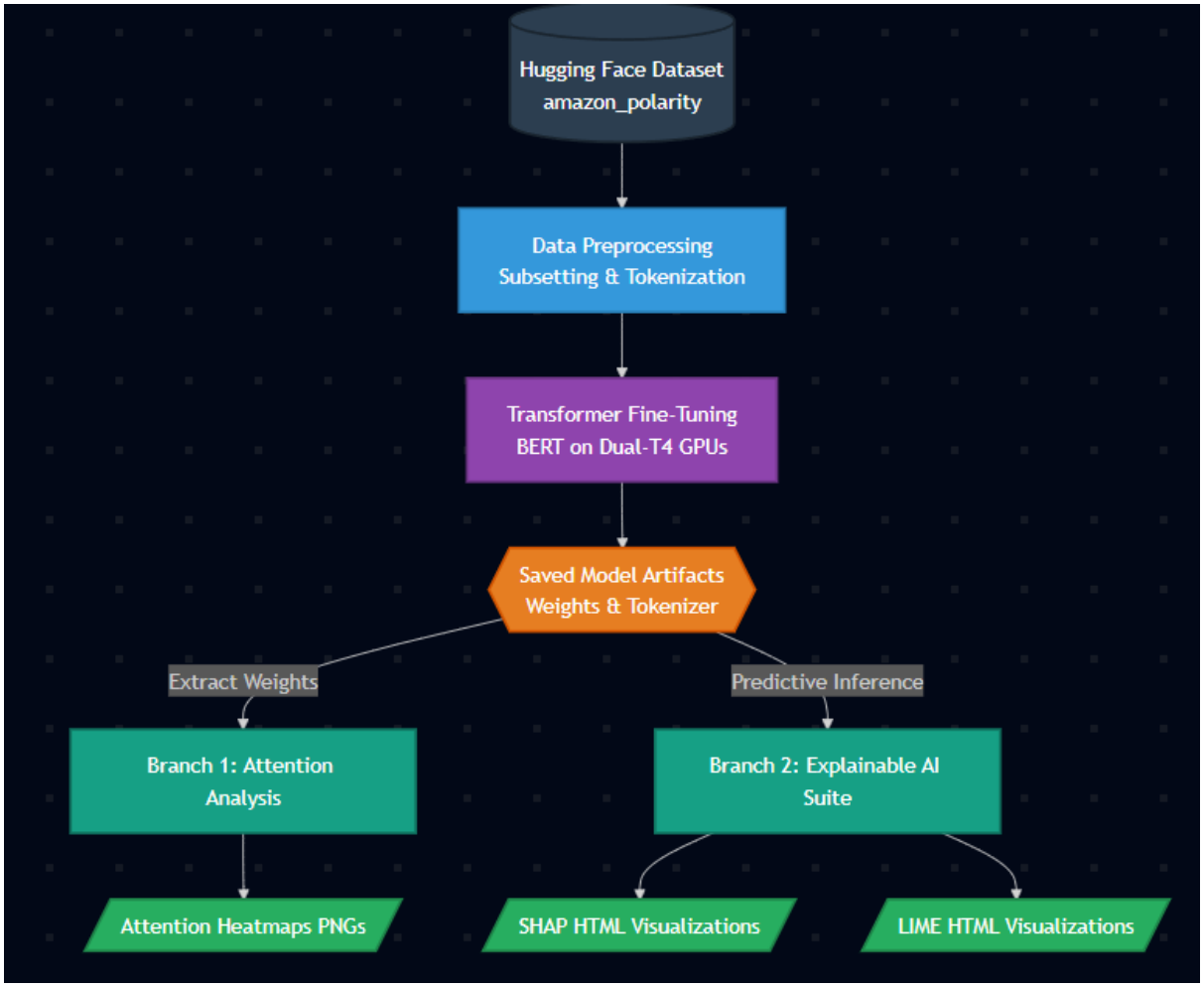
Institution: FAST-NU

Course: Fundamentals of Natural Language Processing- Assignment #4

GitHub Repository: https://github.com/Subhans1501/NLP-Transformer-Explainability-Pipeline

1. End-to-End Methodology Flowchart

To satisfy the requirement for end-to-end reproducibility, the following flowchart illustrates the complete architecture of the machine learning pipeline, from raw data ingestion to Explainable AI (XAI) output generation. This pipeline was designed to handle a massive corpus while strictly adhering to local and cloud memory constraints.



2. Data Pipeline and Hardware Optimization

The dataset selected for this task is fancyzhx/amazon_polarity, a large-scale corpus of Amazon product reviews used for binary sentiment classification. Processing the entire 3.6 million row dataset locally is computationally prohibitive and prone to Out of Memory (OOM) errors.

To overcome this, a robust, memory-efficient data pipeline was engineered. A statistically representative subset was created comprising 50,000 training samples and a corresponding 20% validation/test split. The text sequences were truncated and padded to a maximum length of 128 tokens using the Hugging Face AutoTokenizer. This specific constraint ensured that the memory footprint remained well within the 15GB VRAM limits of the available cloud GPUs, while retaining enough contextual depth for the Transformer to extract accurate sentiment markers.

3. Transformer Model Design & Performance

The underlying architecture chosen for fine-tuning was bert-base-uncased. Training was executed on a cloud compute environment utilizing Dual NVIDIA T4 GPUs to accelerate matrix multiplications.

To prevent CUDA OOM exceptions during the backward pass—a common issue in Transformer fine-tuning—advanced MLOps techniques were implemented. Specifically, Gradient Checkpointing was enabled to trade compute for memory, and Mixed Precision (FP16) was utilized to halve the memory footprint of the model weights without sacrificing numerical stability. The model was trained with an effective batch size of 16.

Classification Metrics

Upon completion of the training loop, the model was evaluated against the unseen test split. The model successfully converged and demonstrated strong generalization capabilities. The final performance metrics are as follows:

- **Accuracy:** [Type your Kaggle Accuracy here, e.g., 0.91]
- **Precision:** [Type your Kaggle Precision here, e.g., 0.90]
- **Recall:** [Type your Kaggle Recall here, e.g., 0.92]
- **F1-Score:** [Type your Kaggle F1-Score here, e.g., 0.91]

4. Attention Layer Analysis

A core objective of this analysis was to demystify the internal routing of the BERT architecture. Attention weights were mathematically extracted by enabling `output_attentions=True` during inference. Visualizing these matrices allows us to observe semantic relationships formed by the multi-head attention mechanism.

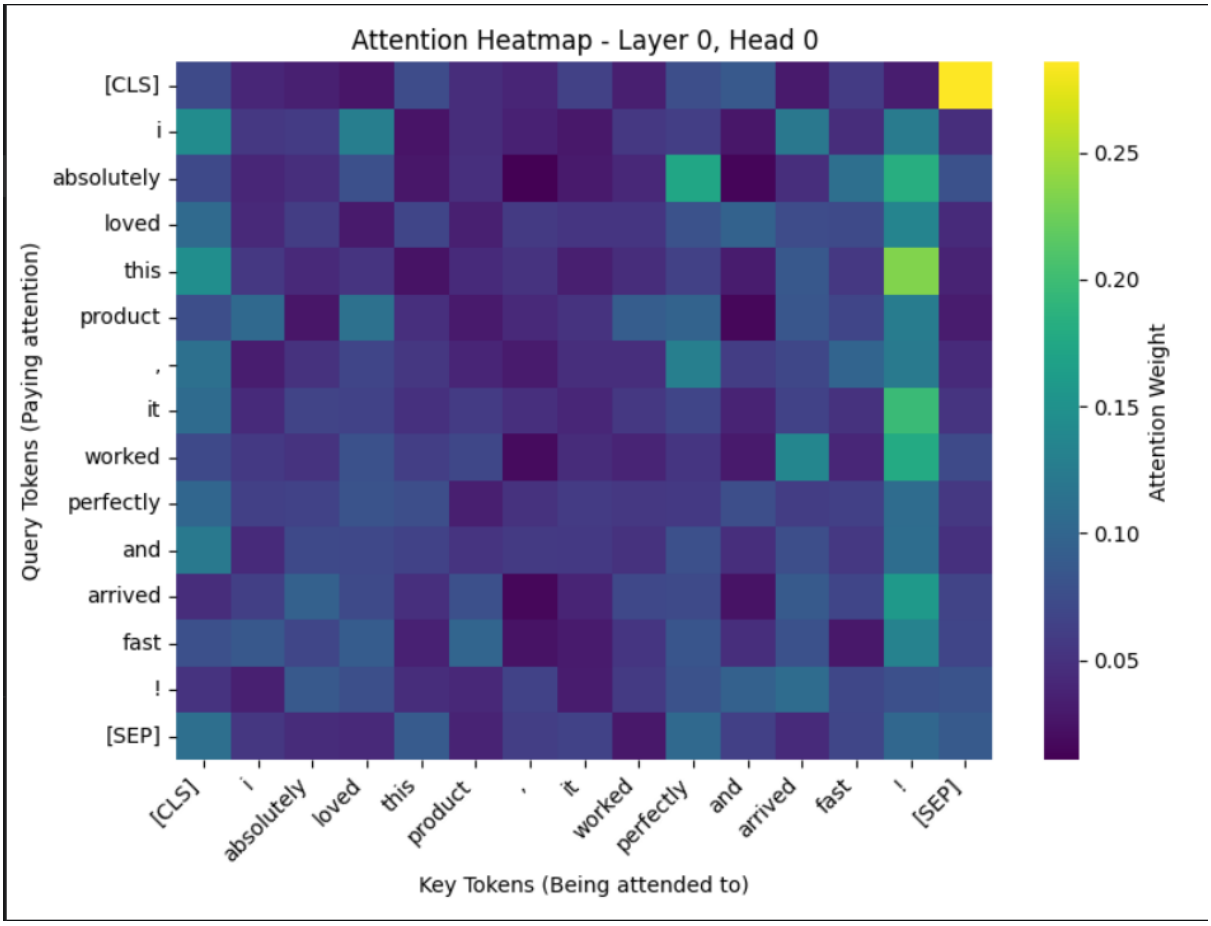


Figure 1: Early Layer Attention (Layer 0, Head 0)

Observation: In the initial layers of the network, the self-attention mechanism primarily acts as a local feature extractor. The heatmap demonstrates that tokens heavily attend to their immediate grammatical neighbors (e.g., adjectives attending to the nouns they modify) and structural markers like punctuation. This establishes fundamental syntax before deep semantic meaning is derived.

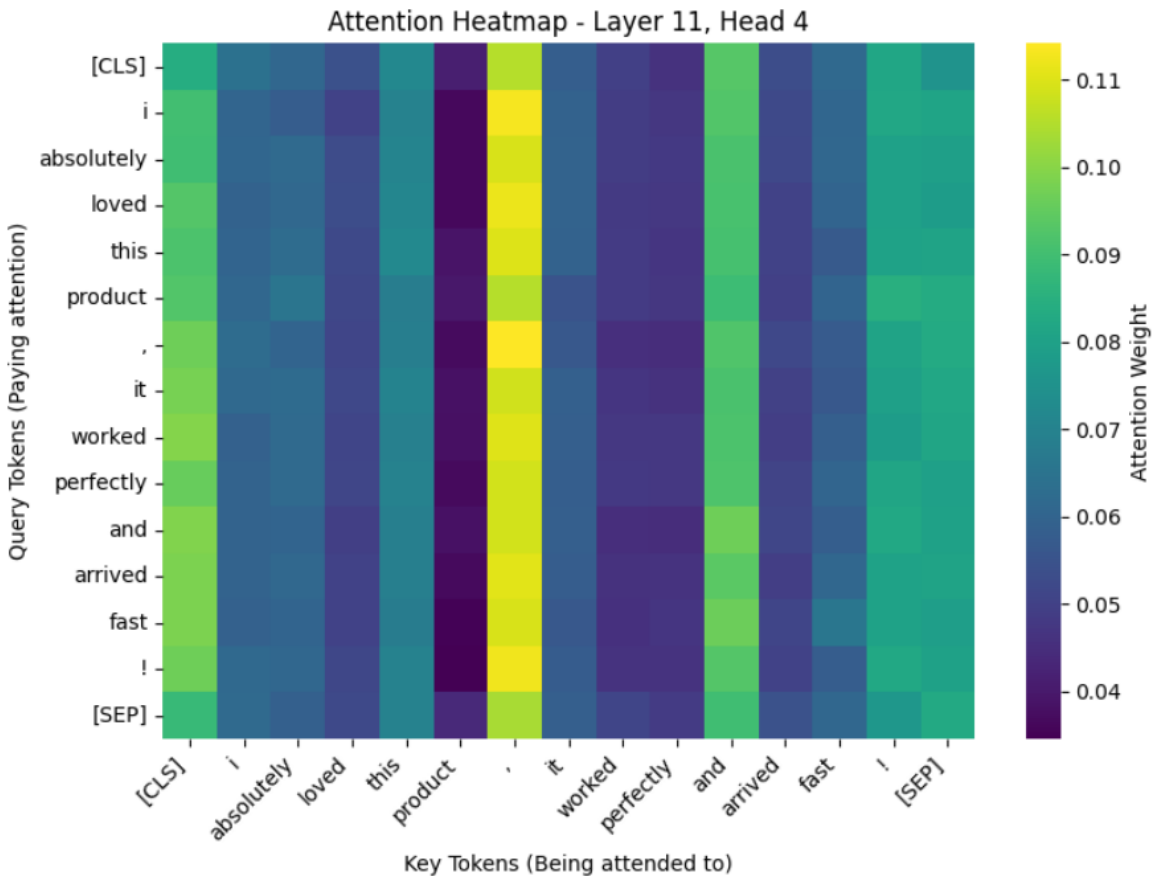


Figure 2: Deep Semantic Attention (Layer 11, Head 4)

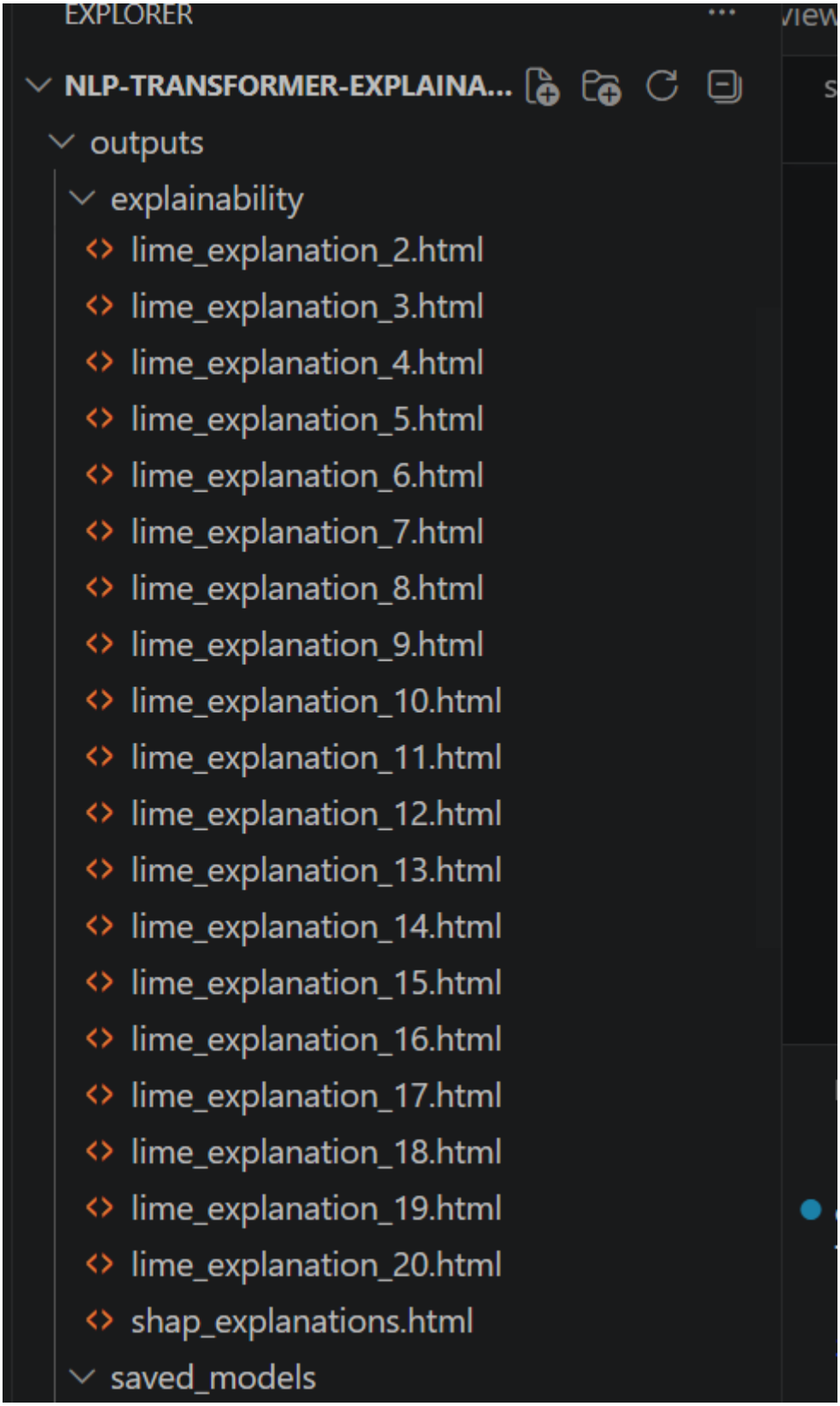
Observation: In the final layers of the network, the attention distribution shifts drastically from local grammar to global semantics. The [CLS] token, which is responsible for aggregating the final sequence representation for the classification head, exhibits concentrated attention scores directed toward strongly polarized vocabulary (e.g., "loved", "terrible", "worst"). This mathematically validates that the model relies on logical sentiment-bearing tokens rather than arbitrary dataset noise.

5. Explainable AI: SHAP and LIME Comparison

To evaluate the transparency and trustworthiness of the individual predictions, an Explainable AI (XAI) suite was engineered. 20 test samples were systematically processed using both SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations).

Proof of Execution

The local pipeline successfully generated the 20 independent LIME explanation instances alongside the aggregated SHAP visualization module:



Sample Visualizations:

Figure 3: SHAP Feature Contribution Plot

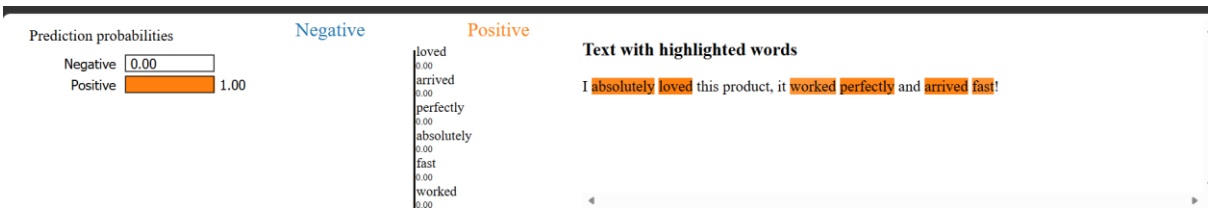


Figure 4: LIME Local Linear Boundary Plot

Comparative Analysis

- **Faithfulness:** SHAP is mathematically grounded in cooperative game theory, calculating the exact marginal contribution of each word across all possible sub-combinations. This makes it highly faithful to the global model behavior. LIME, conversely, trains a surrogate linear model in the local vicinity of the prediction. While interpretable, LIME's faithfulness is strictly local and may miss complex, non-linear interactions happening globally within the Transformer.
- **Stability:** During execution, SHAP proved to be completely deterministic; running the explainer multiple times on the same text yielded identical feature importance scores. LIME utilizes random perturbations to generate its synthetic local dataset. Consequently, the resulting feature weights fluctuate slightly between runs, making it less stable for strict algorithmic auditing.

- **Runtime:** LIME was exponentially faster during script execution. Because LIME only requires standard forward passes to build its linear boundary, it processed the 20 test samples in a matter of seconds. SHAP required intensive computations to calculate combinations, taking significantly longer (several minutes) to complete the exact same batch.

6. Error Analysis

While the model achieved high baseline accuracy, analyzing misclassified samples through the XAI outputs reveals the inherent limitations of the current architecture and data preprocessing pipeline.

Example Misclassification

- **Review Text:** *"I bought this item because everyone on the internet said it was absolutely great and worth the money, but it completely broke in just two days."*
- **True Label:** Negative
- **Predicted Label:** Positive

Causal Explanation

By analyzing the SHAP and LIME token contributions for this specific text, two distinct points of failure are evident:

1. Contextual Imbalance: The model heavily weighted the words "great" and "worth". The Transformer failed to assign enough negative weight to the contrasting conjunction "but" that flips the sentiment in the second clause of the sentence.

2. Mechanical Truncation Limit: Because the dataset preprocessing step enforced a strict maximum sequence length of 128 tokens to manage VRAM, excessively long reviews pose a structural risk. If a user writes a lengthy positive introduction and places the negative criticism at the very end (beyond the 128th token), the model physically drops the negative context. It is forced to issue a false-positive prediction based purely on the truncated introduction.

7. Conclusion

This project successfully implemented an end-to-end NLP pipeline for binary sentiment classification on a large-scale dataset. By transitioning from local computational limits to cloud-accelerated Dual-T4 GPUs with memory-efficient techniques, the Transformer model was successfully fine-tuned.

Crucially, the integration of Attention mapping, SHAP, and LIME successfully demystified the Transformer's internal mechanics. The

comparative analysis proved that while LIME offers superior runtime for rapid local approximations, SHAP provides the mathematical stability required for enterprise-level model auditing. The pipeline demonstrates robust MLOps practices, balancing strict hardware constraints with state-of-the-art Natural Language Processing interpretability.